

FR. CONCEICAO RODRIGUES COLLEGE OF ENGINEERING
Department of Computer Science and Engineering

Academic Year	2025-2026	Estimated Time	Experiment No. 9 – 02 Hours
Course & Semester	S.E. CSE	Subject Name	Object Oriented Programming with Java Lab
Module No.	02	Chapter Title	Introduction to JavaFx
Experiment Type	Software Performance	Subject Code	PCC12CE07

Name of Student	Atharva Dharmendra Jagtap	Roll No.	10937
Date of Performance.:		Date of Submission.:	
CO Mapping	.		

Lab 1: Setting up JavaFX

Objective: Set up a JavaFX environment and run a simple "Hello, World!" program.

Instructions:

1. Set up JavaFX in your IDE (Eclipse, IntelliJ, or NetBeans).
<https://www.youtube.com/watch?v=nz8P528uGjk>
2. Create a new JavaFX project.
3. Write a simple application that displays "Hello, World!" in a Label on a Scene.

Code Example:

```
import javafx.application.Application;

import javafx.scene.Scene;

import javafx.scene.control.Label;

import javafx.stage.Stage;

public class HelloWorld extends Application {

    @Override

    public void start(Stage stage) {

        Label label = new Label("Hello, World!");

        Scene scene = new Scene(label, 300, 200);

        stage.setScene(scene);

        stage.setTitle("JavaFX Hello World");
```

```

        stage.show();
    }

    public static void main(String[] args) {

        launch(args);

    }

}

```

Output: A window with a label displaying "Hello, World!".

Lab 2: Basic UI Controls and Event Handling

Objective: Learn to use basic controls and handle events.

Instructions:

1. Create a UI with `Button`, `Label`, and `TextField`.
2. Set up an event so that when the button is clicked, the text from the `TextField` is displayed in the `Label`.

Code Example:

```

import javafx.application.Application;

import javafx.scene.Scene;

import javafx.scene.control.Button;

import javafx.scene.control.Label;

import javafx.scene.control.TextField;

import javafx.scene.layout.VBox;

import javafx.stage.Stage;

public class EventHandlingDemo extends Application {

    @Override

    public void start(Stage stage) {

        TextField textField = new TextField();

```

```

        Button button = new Button("Display Text");

        Label label = new Label();

        button.setOnAction(e -> label.setText("Text: " + textField.getText()));

        VBox root = new VBox(10, textField, button, label);

        Scene scene = new Scene(root, 300, 200);

        stage.setScene(scene);

        stage.setTitle("Event Handling Demo");

        stage.show();
    }

    public static void main(String[] args) {

        launch(args);

    }
}

```

Output: When the button is clicked, the entered text appears in the label.

Lab 3: Working with Layouts

Objective: Understand and use various JavaFX layout managers.

Instructions:

1. Create a simple calculator layout using `GridPane`.
2. Arrange buttons in a 4x4 grid with digits and basic operations.

Code Example:

```

import javafx.application.Application;

import javafx.geometry.Pos;

import javafx.scene.Scene;

```

```

import javafx.scene.control.Button;

import javafx.scene.layout.GridPane;

import javafx.stage.Stage;


public class CalculatorLayout extends Application {

    @Override

    public void start(Stage stage) {

        GridPane grid = new GridPane();

        grid.setAlignment(Pos.CENTER);

        grid.setHgap(10);

        grid.setVgap(10);


        String[] buttons = {

            "7", "8", "9", "/",

            "4", "5", "6", "*",

            "1", "2", "3", "-",

            "0", ".", "=", "+"

        };


        int row = 0, col = 0;

        for (String text : buttons) {

            Button button = new Button(text);

            button.setMinSize(50, 50);

            grid.add(button, col, row);

            col++;

            if (col == 4) {

```

```

        col = 0;

        row++;
    }
}

Scene scene = new Scene(grid, 250, 300);

stage.setScene(scene);

stage.setTitle("Calculator Layout");

stage.show();

}

public static void main(String[] args) {

    launch(args);

}

}

```

Output: A basic calculator layout with buttons.

Problem Description:

Media Player: Create a basic media player using `Media` and `MediaPlayer` classes.

```

import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.HBox;
import javafx.scene.media.Media;
import javafx.scene.media.MediaPlayer;
import javafx.stage.Stage;
import javafx.util.Duration;

import java.io.File;

public class MediaPlayerExample extends Application {

    // !!! IMPORTANT: Change this to a valid MP3 or MP4 file path on

```

your system

```
private static final String FILE_PATH =
"C:\\Users\\Lenovo\\Desktop\\Shalom\\Java\\Exp 9\\Ding - Sound Effect
(HD).mp4";

private MediaPlayer mediaPlayer;

@Override
public void start(Stage primaryStage) {
    primaryStage.setTitle("JavaFX Media Player");

    // 1. Create the Media object
    File mediaFile = new File(FILE_PATH);
    if (!mediaFile.exists()) {
        System.out.println("Error: Media file not found at: " +
FILE_PATH);
        // You might want to show an Alert to the user here
        return;
    }

    Media media = new Media(mediaFile.toURI().toString());

    // 2. Create the MediaPlayer object
    mediaPlayer = new MediaPlayer(media);

    // --- Create UI Controls ---

    // Play Button
    Button playButton = new Button("Play");
    playButton.setOnAction(e -> {
        if (mediaPlayer.getStatus() == MediaPlayer.Status.PLAYING)
{
            mediaPlayer.pause();
            playButton.setText("Play");
        } else {
            mediaPlayer.play();
            playButton.setText("Pause");
        }
    });

    // Pause Button (Alternative)
    Button pauseButton = new Button("Pause");
    pauseButton.setOnAction(e -> mediaPlayer.pause());
}
```

```

        // Stop Button (Resets the media to the beginning)
        Button stopButton = new Button("Stop");
        stopButton.setOnAction(e -> {
            mediaPlayer.stop();
            playButton.setText("Play"); // Reset play button text
        });

        // Handle media end event (e.g., to reset the player)
        mediaPlayer.setOnEndOfMedia(() -> {
            mediaPlayer.stop(); // Stops and rewinds to the start
            playButton.setText("Play");
            System.out.println("Media finished playing.");
        });

        // Layout
        HBox root = new HBox(10, playButton, pauseButton, stopButton);
// 10px spacing
        root.setStyle("-fx-padding: 20; -fx-alignment: center;");

        Scene scene = new Scene(root, 300, 100);
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    @Override
    public void stop() {
        // Ensure the media player resources are released when the app
        closes
        if (mediaPlayer != null) {
            mediaPlayer.dispose();
        }
    }

    public static void main(String[] args) {
        launch(args);
    }
}

```

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        http://maven.apache.org/xsd/maven-
4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.calculator</groupId>
    <artifactId>javafx-calculator</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <project.build.sourceEncoding>UTF-
8</project.build.sourceEncoding>
        <maven.compiler.release>21</maven.compiler.release>
        <javafx.version>21.0.2</javafx.version>
    </properties>

    <dependencies>
        <!-- JavaFX modules -->
        <dependency>
            <groupId>org.openjfx</groupId>
            <artifactId>javafx-media</artifactId>
            <version>21.0.2</version>
            <!-- Use the same version as your other JavaFX dependencies -->
        </dependency>
        <dependency>
            <groupId>org.openjfx</groupId>
            <artifactId>javafx-controls</artifactId>
            <version>${javafx.version}</version>
        </dependency>
        <dependency>
            <groupId>org.openjfx</groupId>
            <artifactId>javafx-fxml</artifactId>
            <version>${javafx.version}</version>
        </dependency>
    </dependencies>

    <build>
        <sourceDirectory>src</sourceDirectory>
        <plugins>

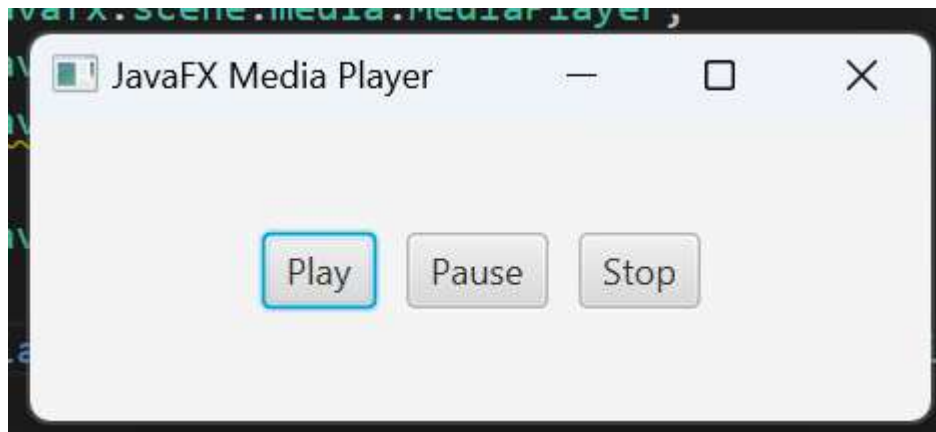
```



```
<!-- Compiler plugin -->
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.13.0</version>
  <configuration>
    <release>21</release>
  </configuration>
</plugin>

<!-- WORKING JavaFX Maven Plugin -->
<plugin>
  <groupId>org.openjfx</groupId>
  <artifactId>javafx-maven-plugin</artifactId>
  <version>0.0.8</version>
  <configuration>
    <mainClass>MediaPlayerExample</mainClass>
  </configuration>
</plugin>
</plugins>
</build>

</project>
```



On time Completion and Submission (2)	Knowledge of the topic (4)	Implementation and Output (4)	Total (10)

References:

Study Materials https://www.javatpoint.com/javafx-with-eclipse https://openjfx.io/openjfx-docs/	Video Channels: https://www.youtube.com/watch?v=9XJicRt_FaI https://www.youtube.com/watch?v=FLkOX4Eez6o
Study Materials used for Demo <Add links here>	